[Huyen Nguyen] Process Fundamentals, Memory Layout

1. Process Identifiers, fork()/vfork()

1.1. What is fork()?

It's a system call that creates a child process — an almost identical copy of the calling parent process.

After fork() executes, 2 processes exist simultaneously and both continue running from the line right after fork().

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child: PID=%d, PPID=%d\n", getpid(), getppid());
    } else {
        printf("Parent: PID=%d, child=%d\n", getpid(), pid);
    }
    return 0;
}
```

Return value	Who receives it	Meaning
< 0	Parent process	Child creation failed (out of memory, process limit exceeded...)

Return value	Who receives it	Meaning
== 0	Child process	This is the child; pid equals 0 here
>0	Parent process	This value is the PID of the child that was just created

The print order between parent/child is non-deterministic — it's decided by the OS scheduler, not by the code.

Both processes execute their own separate return 0; — they are two independent processes from the moment fork() is called onward (using copy-on-write memory, not sharing variables).

1.2. How does vfork() differ from fork()?

vfork() also creates a child process, but there are two core differences that make it far more dangerous:

- No memory copying: The child process shares the exact same address space as the parent. This means every variable and memory region exists in the same location in RAM.
- Parent process is suspended: The parent process will not resume execution until the child calls exec() or terminates via _exit(). This is a major shift from fork(), where both parent and child run concurrently right away.
-

The dangerous point

```
#include <unistd.h>
#include <stdio.h>
int main() {
    int x = 100;
    pid_t pid = vfork();
    if (pid == 0) {
        x = 999;           /* DANGEROUS: modifying a variable shared with the parent*/
        _exit(0);           /* MUST use _exit(), not exit()*/
    }
    printf("Parent: x = %d\n", x);  /* x was changed by the child -> prints 999, not 100*/
    return 0;
}
```

-> The parent program's data can be corrupted in unpredictable ways, especially in complex applications with shared variables, pointers, and stack space.

1.3. Note

- Forgetting to check whether fork() returns -1 (resource exhaustion error).

- Modifying a variable in the child of vfork() before calling exec()/_exit() → corrupts the parent's data.
- Using exit() (instead of _exit()) in the child of vfork() → flushes I/O buffers twice → prints duplicate data.

2. exit Functions and Process Termination

2.1. Exit Status Truncation (8-bit Exit Code)

Only the low 8 bits of the exit status are meaningful (0–255); exit(300) actually returns 44 (300 % 256).

child.c:

```
#include <stdlib.h>
int main() {
    exit(300);   /* 300 % 256 = 44*/
}
```

parent.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        execl("./child", "child", NULL);
        _exit(127); /* if execl fails*/
    } else {
        int status;
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Exit status received: %d\n", WEXITSTATUS(status));
        }
    }
    return 0;
}
```

exit(300) — but the exit status in Unix/Linux only stores the low 8 bits (0–255), in the status field passed to wait()/waitpid(), or in the shell's $? variable.

exit() called with		Actual value received ($? or WEXITSTATUS)
exit(0)		0
exit(255)		255
exit(256)		0 (256 % 256 = 0)
exit(257)		1
exit(-1)		255 (-1 as an 8-bit unsigned value is 0xFF)

300 in binary is 1 0010 1100. Taking the low 8 bits: 0010 1100 = 44. Using _exit() in a normal child process (after a regular fork()) but still expecting the output to be flushed.

If a program has multiple kinds of errors and you plan to use exit codes to distinguish them -> keep the value in the range 0–255, conventionally 1–125 (126 and above is often used by the shell for special meanings, such as "command not found" or "killed by signal").

2.2. Process Termination

Functions	Flush buffer?	Runs atexit handlers?
`return` from `main()`	Yes	Yes
`exit()`	No	No
`_exit()`	**No**	**No**

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
void cleanup() { printf("Cleaning up before exit\n"); }
int main() {
    atexit(cleanup);
    printf("Running...");
    if (fork() == 0) {
        _exit(0);              /* Child: does NOT flush "Running...", does not call
cleanup()*/
    }
    exit(0);                   /* Parent: flushes buffer, calls cleanup()*/
}
```

2.3. Zombie process (PID Exhaustion & Server Hangs)

fork() in a loop while forgetting to call wait() ?

-> After running for a few days, this spawns tens of thousands of zombies, completely exhausting the PID table → the system crashes, unable to run any further commands (fork: Resource temporarily unavailable)

Fix: Set up a handler that catches SIGCHLD to automatically clean up as children die:

```
#include <signal.h>
#include <sys/wait.h>

void sigchld_handler(int s) {
    while (waitpid(-1, NULL, WNOHANG) > 0);
}

/* In main()*/
signal(SIGCHLD, sigchld_handler);
```

Trying to use kill -9 to destroy a zombie

The zombie is already dead — it can no longer receive any signal whatsoever.

You must kill the zombie's parent process (or fix the parent's code to call wait()).

2.4. Orphan process

Scenario: The parent process hits a memory error (segmentation fault) and dies suddenly. Dozens of child processes keep running in the background, performing heavy computation loops or holding database connections open.

What happens to PID and PPID when the parent terminates?

- PID (Process ID): Remains unchanged.
- PPID: Immediately updates. The kernel re-parents the child process to init (or systemd in modern distributions), changing its PPID to 1 (or the PID of the system's subreaper process).
- Signal: The OS does not send any default signal to inform the child process that its parent has died.

Consequence: System resource leaks (memory leaks, DB connection leaks) that doesn't notice.

Fix: In the child process, periodically check whether the original parent is still alive:

```
if (getppid() == 1 || getppid() != original_parent_pid) {
    /* The original parent is gone -> child proactively exits*/
    exit(0);
}
```

Or use the prctl(PR_SET_PDEATHSIG, SIGTERM) flag on Linux (the OS will automatically send a termination signal to the child when the parent dies).

2.5. Daemon process

What is a Daemon process?

A daemon is a special background process that runs continuously throughout the system's lifecycle without a user interface (UI/CLI) and without being attached to any terminal session.

In Linux/Unix systems, daemon names usually end with the letter "d" (e.g., sshd, httpd, systemd, crond).

Why do we need Daemon processes?

- Provide continuous infrastructure services: Run Web Servers, Databases, and SSH Services to handle requests 24/7.
- Perform scheduled/automated tasks: Manage cron jobs, monitor system logs, or clean up cache without manual user intervention.
- Independence from user sessions: When logged in via SSH, launching a standard program will cause it to stop if the terminal closes or the network drops. A daemon resolves this by completely detaching from the controlling terminal.

Standard procedure for creating a Daemon Process

```
void create_daemon() {
    pid_t pid = fork();
    /* First fork: Parent exits, child continues running in background */
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS); /* Parent process exits */
    /* Create a new session: Detach from the controlling terminal */
    if (setsid() < 0) exit(EXIT_FAILURE);
    /* Second fork: Ensure the daemon never re-acquires a terminal */
    pid = fork();
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS);
    /* Reset file mode creation mask (umask) */
    umask(0);
```

```
    /* Change working directory to root "/" */
    chdir("/");
    /* Close default File Descriptors and redirect standard streams */
    close(STDIN_FILENO);
    close(STDOUT_FILENO);
    close(STDERR_FILENO);

    freopen("/dev/null", "r", stdin);
    freopen("/var/log/my_daemon.log", "a", stdout);
    freopen("/var/log/my_daemon_err.log", "a", stderr);
}
int main() {
    create_daemon();
    /* Main daemon execution loop */
    while (1) {
        printf("Daemon is running safely in background...\n");
        fflush(stdout);
        sleep(5);
    }
    return 0;
}
```

Forgetting to call setsid()?

Delude the program is running in the background, but when log out or close the terminal

-> The daemon is immediately killed because it receives SIGHUP.

Forgetting to redirect the standard input/output/error streams (stdin, stdout, stderr)?

->The daemon still tries to print to the screen with printf(). When the original terminal is closed, this write can trigger a fatal SIGPIPE error. ->Or, its output ends up printing over another user's terminal screen!

Fix: Redirect all output to a log file or /dev/null

```
freopen("/dev/null", "r", stdin);
freopen("app.log", "a", stdout);
freopen("error.log", "a", stderr);
```

- stdin (Standard Input): Receives data when you type on the keyboard (the scanf function).
- stdout (Standard Output): Pumps text out to the terminal screen when you call printf().

- stderr (Standard Error): Pumps error messages out to the terminal screen.

3. wait, waitpid, and waitid Functions

3.1. Using the status variable directly?

```
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child: I'm calling exit(1)\n");
        exit(1);
    } else {
        int status;
        wait(&status);

        printf("1. Printing status directly:        status = %d  \n", status);
        printf("2. Using macro WEXITSTATUS(status): exit_code = %d  \n", WEXITSTATUS(status));
    }

    return 0;
}
```

Program output:

```
1. Printing status directly:        status = 256
2. Using macro WEXITSTATUS(status): exit_code = 1
```

-> When printing the status variable directly, the program doesn't report an error, but the printed value is wrong.

HIGH BYTE (Bits 8→15): Stores the exit() code

LOW BYTE (Bits 0→7): Stores the signal number that caused termination

In the C library, the WEXITSTATUS macro is defined very simply as a right bit-shift by 8 (>> 8).

3.2. "Signal coalescing" when working with SIGCHLD?

In the Linux kernel, the SIGCHLD signal is not queued:

```
void sigchld_handler(int s) {
    waitpid(-1, NULL, WNOHANG); /* only sends 1 signal to the parent; the remaining child processes become zombies*/
}
```

```
void sigchld_handler(int s) {
    while (waitpid(-1, NULL, WNOHANG) > 0); /* use a loop so that all child processes get reaped*/
}
```

4. Race Condition between fork/exec/wait

4.1. Child dies before the Parent manages to save the PID?

This is a bug when writing command-line tools (Shell/Bash) or process management systems:

```
pid_t child_pid = fork();
if (child_pid == 0) {
    exit(0); /* Child runs fast and exits immediately*/
} else {
    add_child_to_management_list(child_pid);
}
```

The child runs fast → dies immediately → sends SIGCHLD to the Parent.

The Parent's sigchld_handler runs → tries to find the child in the list to remove it → Not found (because the Parent hasn't added it yet)

Afterward, the Parent then reaches the line add_child_to_management_list → This already-dead child gets permanently stuck in the management list

Failing to save the PID in time creates a "ghost process" that gets permanently stuck in the software, corrupting status information and gradually causing a memory overflow in the application's management structures.

4.2. Child runs before the Parent manages to prepare resources

Case 1: Not checking if (f == NULL)

```
pid_t pid = fork();
if (pid == 0) {
   FILE *f = fopen("data.txt", "r");
   /* If the CPU schedules the Child to run first, "data.txt" HASN'T BEEN CREATED YET -> fopen
returns NULL -> Child crashes*/
} else {
   FILE *f = fopen("data.txt", "w");
   fprintf(f, "Important data\n");
   fclose(f);
}
```

If the Child performs a read operation right after the fopen line (such as fgets, fscanf, or fgetc(f))

-> The Child will read directly through a NULL pointer. -> The operating system immediately raises a SIGSEGV signal, crashing the program with the error: Segmentation fault

Case 2: Checking if (f == NULL)

```
FILE *f = fopen("data.txt", "r");
if (f == NULL) {
   perror("Error opening file");
   exit(1);
}
```

-> Error reported: "Error opening file: No such file or directory"

5. The system Function

```
int main() {
   printf("1. Listing files:\n");
   system("ls -l");
   printf("\n2. Creating a new directory:\n");
   system("mkdir NewFolder");
   printf("\n3. Checking the system date/time:\n");
   system("date");
   return 0;
}
```

The system("ls") function automatically does:

1. Calls fork() —> Creates a child process

2. Calls exec() —> Launches a Shell (/bin/sh) to run the command "ls"
3. Calls waitpid() —> Waits for the command to finish before returning the result

Advantages: Convenient and concise

- To run any Linux command (like clear, ping, mkdir...), just need to type that exact command name inside the quotation marks.

Disadvantages: Slow and resource-intensive

- Each time system() is called, the computer has to launch an entire intermediary Shell program (/bin/sh).

6. Process Groups, Sessions, Controlling Terminal & Job Control

6.1. Process Groups

A Process Group in Linux/Unix is a collection of one or more related processes managed by the OS as a single unified entity.

PGID (Process Group ID): Each process group is identified by a unique positive integer known as the PGID.

Process Group Leader:

- The process that creates the group.
- At the time of group creation, this process has PID == PGID.
- The Process Group continues to exist as long as at least one process remains in the group (even after the Leader has terminated).

Why do we need Process Groups?

- Job Control in Shell: Allows the shell to manage foreground and background jobs as single units.
- Signal Distribution: Enables sending signals (e.g., SIGINT, SIGKILL) to an entire group of processes simultaneously.
- Daemon Process Creation: Helps detach processes from controlling terminals to create independent background services.

System Calls for Working with Process Groups

Function	System Call Description

pid_t getpgrp(void)	Returns the PGID of the calling process.
pid_t getpgid(pid_t pid)	Returns the PGID of the process specified by pid (pid = 0 refers to the calling process).
int setpgid(pid_t pid, pid_t pgid)	Creates a new Session and a new Process Group (the calling process becomes both the Session Leader and Process Group Leader).
kill(-pgid, SIG...)	Sends a signal to all processes inside process group pgid (indicated by the negative sign).

```
int main() {
   pid_t pid = getpid();
   pid_t pgid = getpgrp(); /* Or getpgid(0)*/
   printf("PID: %d, PGID: %d\n", pid, pgid);
   if (pid == pgid) {
       printf("-> This is Process Group Leader!\n");
   } else {
       printf("-> This isn't Process Group Leader.\n");
   }
   return 0;
}
```

6.2. Sessions

Structure and Components of a Session

A Session in Linux/Unix is the highest level of management in the process hierarchy, consisting of one or more Process Groups.

- SID (Session ID): Each session has a unique identification number.
- Session Leader: The process that initiates the session. Has its Process ID equal to the Session ID (PID = SID)
- Controlling Terminal: A session can have at most one controlling terminal (or none at all). This terminal communicates directly with only one Foreground Process Group at any given time.

Roles of a Session Login and Terminal Management

- When logging into Linux, the operating system creates a new session.
- The shell program (such as /bin/bash) becomes the Session Leader.

Handling Terminal Closure

- The terminal driver detects a disconnection and sends the SIGHUP signal to the Session Leader.

- The Session Leader forwards the SIGHUP signal to all child processes in the session to terminate them.

Sessions in C Programming & Creating Daemons (setsid)

The setsid() function performs three main tasks:

- Creates a new session.
- Creates a new process group.
- Completely severs the link with the old controlling terminal (preventing the process from being terminated when the terminal closes).

Rule when calling setsid(): The calling process must not be a Process Group Leader. Therefore, when programming a Daemon, we must always fork() first, let the parent process terminate (exit(0)), and then have the child process call setsid().

System Call	Description
pid_t getsid(pid_t pid)	Retrieves SID of the specified process pid
pid_t setsid(void)	Creates a new session and detaches the process from the controlling terminal

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    printf("BEFORE FORK (PARENT PROCESS)\n");
    printf("Parent: PID=%d | PGID=%d | SID=%d | TTY=%s\n\n",
           getpid(), getpgrp(), getsid(0), ttyname(STDIN_FILENO));
    if (pid > 0) {
        exit(0);
    }
    if (pid == 0) {
        printf("CHILD PROCESS: BEFORE CALLING setsid()\n");
        printf("Child: PID=%d | PGID=%d | SID=%d | TTY=%s\n\n",
               getpid(), getpgrp(), getsid(0), ttyname(STDIN_FILENO));
        setsid();
        printf("CHILD PROCESS: AFTER CALLING setsid()\n");
        printf("Child: PID=%d | PGID=%d | SID=%d | TTY=%s\n",
               getpid(), getpgrp(), getsid(0),
               ttyname(STDIN_FILENO) ? ttyname(STDIN_FILENO) : "(No TTY left)");
```

```
    }
    return 0;
}
```

Program output:

```
BEFORE FORK (PARENT PROCESS) /*The parent shares Session 4120 of the Terminal*/
    pid_t pid = fork();
Parent: PID=6001 | PGID=6001 | SID=4120 | TTY=/dev/pts/0

CHILD PROCESS: BEFORE CALLING setsid()/*The child still inherits Session 4120 and the TTY from the parent*/
Child: PID=6002 | PGID=6001 | SID=4120 | TTY=/dev/pts/0

CHILD PROCESS: AFTER CALLING setsid()/*The child has become the Session Leader of the new session 6002, completely independent*/
Child: PID=6002 | PGID=6002 | SID=6002 | TTY=(No TTY left)
```

6.3. What í a COntrolling Terminal?

It is the single Terminal window granted the "master" right over a Session.

Responsibilities of a Controlling Terminal

- Decides who has the right to receive keyboard input: At any given time, it only allows one process group to read the data type from the keyboard.
- Emits control signals: When press Ctrl + C or Ctrl + Z, the Controlling Terminal sends signals (SIGINT, SIGTSTP) to the program running in the foreground.
- Emits the death signal (SIGHUP): When this Controlling Terminal window is closed -> It immediately sends a SIGHUP signal to request the termination of all processes belonging to its session.

Rule: A Session can have at most one Controlling Terminal (or none at all).

6.4. What is Job Control

It is a multitasking switching feature inside a Terminal window. It is a tool that helps do multiple things at once within a single Terminal command-line window.

Job Control was created to help pause, minimize, run in the background, or bring a program back to the screen without having to shut it down. Every time type a command in the Terminal, the Shell considers it a Job.

Command	Description
Ctrl + Z	Pause and Minimize
Ampersand & at the end of a command (e.g., sleep 60 &)	Run in the background from the start
jobs	Taskbar
fg	Bring back to the screen
bg	Continue running in the background

7. Command Line Arguments and Environment List

7.1. Command Line Arguments

Definition: Arguments passed directly into a program from the Terminal / Shell or when another process invokes the exec() family of functions.

Core Components

- argc (Argument Count): An integer representing the number of arguments passed in. Its value is always >= 1 because the first argument is always the program's name or executable path.
- argv (Argument Vector): An array of pointers to null-terminated strings (char *), terminated by a NULL element.

```
int main(int argc, char *argv[]) {
    printf("Argument count (argc) = %d\n", argc);

    for (int i = 0; i < argc; i++) {
        printf("  -> argv[%d] = %s\n", i, argv[i]);
    }

    return 0;
}
```

Case 1. No extra arguments

```
./demo
```

-> Output

```
Argument count (argc) = 1
  -> argv[0] = ./demo
```

Case 2. Passing additional arguments

```
./demo "C program" Linux
```

-> Output: In this case, there is 1 program name and 2 user-supplied arguments

```
Argument count (argc) = 3
  -> argv[0] = ./demo
  -> argv[1] = C program
  -> argv[2] = Linux
```

Key Use Cases & Benefits

- Eliminates Code Modification: Recompile code less often by supplying configuration dynamically at runtime.
- Enables Automation: Inject input data immediately upon executing the command.
- Supports Flexible Modes: Design multi-mode tools similar to professional production-grade utilities.
- Inter-Process Control: Enables processes to invoke and instruct each other.

```
execve("/bin/ls", argv, NULL); /* Parent process uses the argv array to direct the child process without modifying the child's code*/
```

--> Passing argc and argv makes programs significantly more flexible, reusable, and natively compatible with the Linux command-line ecosystem.

7.2. Environment List

Definition: A collection of KEY=VALUE formatted strings containing configuration data for the system, users, and sessions.

When a process calls fork(), the child process inherits a complete copy of the parent process's environment variables.

Methods for Accessing Environment Variables

- Using the third argument of main (envp)

```
#include <stdio.h>
int main(int argc, char *argv[], char *envp[]) {
    for (int i = 0; envp[i] != NULL; i++) {
```

```
        printf("%s\n", envp[i]);
    }
    return 0;
}
```

*envp[] is an array of pointers that receives the full list of environment variables passed from the operating system to the process when execution begins.

- Using the global variable environ defined in the C library

```
#include <stdio.h>
extern char **environ;
int main(void) {
    for (char **env = environ; *env != NULL; env++) {
        printf("%s\n", *env);
    }
    return 0;
}
```

environ is a global pointer variable provided by the standard C library on Linux/Unix operating systems.

Criteria	envp Parameter	Global Variable environ
Scope	Scope is restricted to main(). Secondary functions require it to be passed explicitly as an argument.	Global scope: Accessible directly from any function or source file within the project.
Dynamic Updates (setenv)	Calling setenv() to modify or add environment variables leaves envp pointing to stale data.	Automatically updates its pointer to reference the updated memory block following a setenv() invocation.

- Using standard C library functions

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(void) {
    char *user = getenv("USER");
    printf("Hello: %s\n", user ? user : "Anonymous");
    char *mode = getenv("APP_MODE");
    if (mode != NULL && strcmp(mode, "DEBUG") == 0) {
        printf("[LOG] Running in DEBUG mode \n");
    } else {
```

```
        printf("[LOG] Running in PRODUCTION mode \n");
    }
    char *port = getenv("PORT");
    printf("App running on PORT: %s\n", port ? port : "8080 ");
    return 0;
}
```

Default Execution

```
$ gcc app_config.c -o app_config
$ ./app_config
```

--> Output

```
Hello: huyen
[LOG] Running in PRODUCTION mode
App running on PORT: 8080
```

Passing Environment Variables Directly via Command Line

```
$ APP_MODE=DEBUG PORT=3000 ./app_config
```

--> Output

```
Hello: huyen
[LOG] Running in DEBUG mode
App running on PORT: 3000
```

8. Memmory Layout of a C Program

9. Shared Libraries

9.1. Static Libraries

Definition: Uses a Static Library (typically with an .a file extension on Linux/Unix). The entire machine code of the library functions is directly copied and embedded into the final executable binary at compile time.

Linker Process at Build Time

- The compiler processes the object file (main.o).
- When a function call is encountered, the linker locates it inside the .a archive file, copies the machine code (0101...) of that function, and embeds it directly into the binary output.

Post-Compilation Behavior: The final executable contains all required dependencies. Even if the .a static library is deleted, the compiled executable will run without error.

Example

- Header File: math_utils.h (Declares the functions provided by the library)

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int tinh_tong(int a, int b);
int tinh_luy_thua(int co_so, int so_mu);
#endif
```

- Library Implementation: math_utils.c (Contains the actual code implementations)

```
#include "math_utils.h"
int tinh_tong(int a, int b) {
    return a + b;
}
int tinh_luy_thua(int co_so, int so_mu) {
    int ket_qua = 1;
    for (int i = 0; i < so_mu; i++) {
        ket_qua *= co_so;
    }
    return ket_qua;
}
```

- Main Application: main.c

```
#include <stdio.h>
#include "math_utils.h"

int main(void) {
    int a = 5, b = 3;
    printf("Sum of %d and %d = %d\n", a, b, tinh_tong(a, b));
    printf("%d power %d = %d\n", a, b, tinh_luy_thua(a, b));
    return 0;
}
```

- Compilation & Archive Pipeline

```
gcc -c math_utils.c -o math_utils.o  # Compile source into machine code object file
ar rcs libmathutils.a math_utils.o    # Pack object file into static library archive
gcc main.c -L. -lmathutils -o app    # Statically link library into executable
```

- Execution Output

```
Sum of 5 and 3 = 8
5 power 3 = 125
```

Advantages

- Complete Independence: The executable can be transferred and run on another machine immediately, without worrying about missing library dependencies on the target system.
- Fast Startup Speed: Eliminates runtime overhead spent searching for and loading external library dependencies upon program launch.

Disadvantages

- Large Executable Size: The binary footprint grows significantly because it embeds the complete machine code of all referenced library dependencies.
- High Memory Consumption: If 10 instances of the process run simultaneously, 10 distinct copies of the library code are loaded into RAM 10 separate times.
- Difficult Maintenance & Updates: If a library bug fix or security patch is released, the entire application must be recompiled from source to incorporate the changes.

9.2. Dynamic Linking

Definition: Uses a Shared Library (typically with a .so file extension on Linux/Unix). At compile time, the linker only embeds references (function names/symbol addresses) into the binary. When the program is launched, the system dynamic linker locates and loads the .so library file into RAM.

Updating Features & Bug Fixes: You only need to replace the .so file with the updated version; all applications relying on that library will automatically use the updated code without requiring a rebuild or recompilation.

Advantages

- Saves Disk Space: Executable files are lightweight because they contain only dynamic function calls and references.
- Saves Memory (Shared Memory): Multiple concurrent processes can share a single copy of the .so file in RAM (hence the name Shared Library).
- Easy Upgrades & Patching: When fixing bugs or patching security issues, simply replace the .so binary—no need to recompile the main application.

Disadvantages

- Dependency Issues: If the target machine is missing the required .so file or has a version mismatch, the program will fail to execute with runtime dynamic link errors.

10. getrlimit and setrlimit Functions (resource limits)

In Linux/Unix operating systems, getrlimit and setrlimit are system calls used to view and set resource consumption limits for a process (such as RAM usage, CPU execution time, number of open file descriptors, etc.).

The primary objective of this mechanism is to prevent a buggy or rogue process from exhausting system resources (for example, infinitely looping, leaking memory, or opening too many files and freezing the host system).

10.1. The struct rlimit Structure

```
struct rlimit {
    rlim_t rlim_cur;  /* Soft limit*/
    rlim_t rlim_max;  /* Hard limit */
};
```

Soft limit (rlim_cur)

- The effective, currently enforced limit for the process.
- Exceeding this threshold causes the OS to return error codes or send signals to terminate the process.
- An unprivileged process can dynamically raise or lower its Soft limit, provided it does not exceed the Hard limit.

Hard limit (rlim_max)

- Acts as the absolute ceiling for the Soft limit.
- Unprivileged processes can only decrease the Hard limit; only the root user or privileged processes (CAP_SYS_RESOURCE) can increase it.

10.2. Key Resource Types

Resource Constant	Meaning	Behavior Upon Exceeding Limit
RLIMIT_NOFILE	Maximum number of open file descriptors (file handles) allowed simultaneously	Calls to open(), socket(), or accept() fail with EMFILE

Resource Constant	Meaning	Behavior Upon Exceeding Limit
RLIMIT_CPU	Maximum CPU execution time allowed in seconds	The kernel delivers a SIGXCPU signal to terminate the process
RLIMIT_AS	Maximum address space size (virtual memory = RAM + Swap) in bytes	Memory allocation calls like malloc() or mmap() fail and return NULL
RLIMIT_FSIZE	Maximum size of a single file the process is allowed to write to disk	The kernel delivers a SIGXFSZ signal to terminate the process
RLIMIT_CORE	Maximum size of a core dump file created when a crash occurs	The dump file is either truncated or omitted entirely

Example: Using RLIMIT_CPU to safeguard against host freezes caused by infinite loops.

```
#include <stdio.h>
#include <sys/resource.h>
#include <unistd.h>
int main(void) {
    struct rlimit rl;
    rl.rlim_cur = 2;
    rl.rlim_max = 5;
    setrlimit(RLIMIT_CPU, &rl);
    printf("Starting infinite loop (kernel will terminate this after 2 CPU seconds)...\n");
    while (1) {
    }
    return 0;
}
```

-> Execution Output

```
Starting infinite loop (kernel will terminate this after 2 CPU seconds)...
CPU time limit exceeded (core dumped)
```

The kernel automatically issues a SIGXCPU signal to abort the process, protecting CPU resources from starvation.

11. ELF format, symbol table, relocation - supplementary linking concepts

11.1. ELF (Executable and Linkable Format)

ELF is the standard binary file format used in Linux/Unix operating systems.

- Relocatable Object File (.o): Unlinked machine code files.
- Executable File (app, zombie_demo): Ready-to-run program files.
- Shared Object File (.so): Dynamic library files.
- Core Dump File: Diagnostic reports generated when a program crashes.

Internal Structure of an ELF File

ELF Header: Contains general system metadata .text: Stores executable machine code .rodata: Stores read-only data .data: Stores initialized global and static variables. .bss: Stores uninitialized global and static variables .symtab: Symbol Table .rel: Relocation Table

11.2. Symbol Table

The Symbol Table acts like a "program directory" embedded inside an ELF file.

It maps out every function name and global variable name, paired with its corresponding location/offset.

Example

Compile object files

```
gcc -c main.c -o main.o
gcc -c math.c -o math.o
```

Inspect the Symbol Table using the nm utility:

```
nm main.o
```

--> Output

```
0000000000000000 T main
                 U phep_cong
```

T indicates that main is defined in the text section with an assigned offset, while U indicates that phep_cong is Undefined

11.3. Relocation

Relocation is the process where the Linker resolves and patches the actual memory addresses of functions and variables into the final executable.

How Relocation Works: When the Linker merges main.o and math.o to produce the final executable app

- The Linker resolves the exact memory address of the target function (e.g., phep_cong at 0x401122).
- The Linker performs Relocation: It locates the placeholder bytes (e.g., 0x00000000) in the instruction code and overwrites them with the true target address (0x401122).
- The function call instruction now contains the resolved target address and is ready for execution.

Example: (Disassembly Output via objdump)

```
0000000000001129 <main>:
   ...
   1138:   e8 0c 00 00 00          call   1149 <phep_cong>
```

The relative offset bytes 0c 00 00 00 represent the resolved address offset pointing directly to the phep_cong function implementation.

12. Reading compiler-generated Assembly (gcc -S / objdump -d) to identify a function's own stack frame

12.1. Commands to View Assembly Code on Linux

- Using gcc -S

```
gcc -S -O0 demo.c -o demo.s
cat demo.s
```

- Using objdump -d

```
gcc -c demo.c -o demo.o
objdump -d demo.o
```

12.2. Comparing C Code and Assembly Code

```
int tinh_toan(int a, int b) {
```

```
    int tong = a + b;
    return tong;
}
```

When running gcc -S -O0 demo.c, GCC generates the following Assembly code:

```
tinh_toan:
    pushq   %rbp              # Push caller's %rbp onto the Stack
    movq    %rsp, %rbp        # Set up new %rbp frame anchor
    subq    $16, %rsp         # Allocate 16 bytes for the Stack Frame
    movl    %edi, -4(%rbp)    # Save 1st parameter (from %edi) into memory location [-4(%rbp)]
    movl    %esi, -8(%rbp)    # Save 2nd parameter (from %esi) into memory location [-8(%rbp)]
    movl    -4(%rbp), %edx    # Load 'a' from [-4(%rbp)] into %edx
    movl    -8(%rbp), %eax    # Load 'b' from [-8(%rbp)] into %eax
    addl    %edx, %eax        # %eax = a + b
    movl    %eax, -12(%rbp)   # Save result into local variable 'tong' at [-12(%rbp)]
    movl    -12(%rbp), %eax   # Move 'tong' value into %eax (return value register)
    leave                     # Clean up 16 bytes, restore previous %rsp and %rbp
    ret                       # Jump back to caller function
```

13. Stack frame layout

13.1. Prologue / Epilogue

Every time a function is called, it creates its own "frame" on the stack to hold local variables, saved values, etc.

Prologue (function entry) — sets up the frame:

```
push %rbp        ;
mov  %rsp, %rbp  ;
sub  $0x20, %rsp ;
```

Epilogue (function exit) — tears down the frame before returning:

```
mov  %rbp, %rsp  ;
pop  %rbp        ;
ret
```

13.2. %rbp vs %rsp

Register	Role
%rsp	Stack pointer-always points to the last memory cell in use, constantly changes with push/pop/call
%rbp	Frame pointer-fixed throughout the function body, used as a reference point to access local variables and stack-passed parameters

13.3. Passing arguments: registers vs. stack

System V AMD64 passes arguments through registers first:

- Integers/pointers (first 6): %rdi, %rsi, %rdx, %rcx, %r8, %r9
- Floating-point (first 8): %xmm0 → %xmm7

If there are more than that → the extra arguments are pushed onto the stack, in right-to-left order, in the memory region above the return address (i.e. from 16(%rbp) upward, since 8(%rbp) is the return address and (%rbp) is the saved rbp).

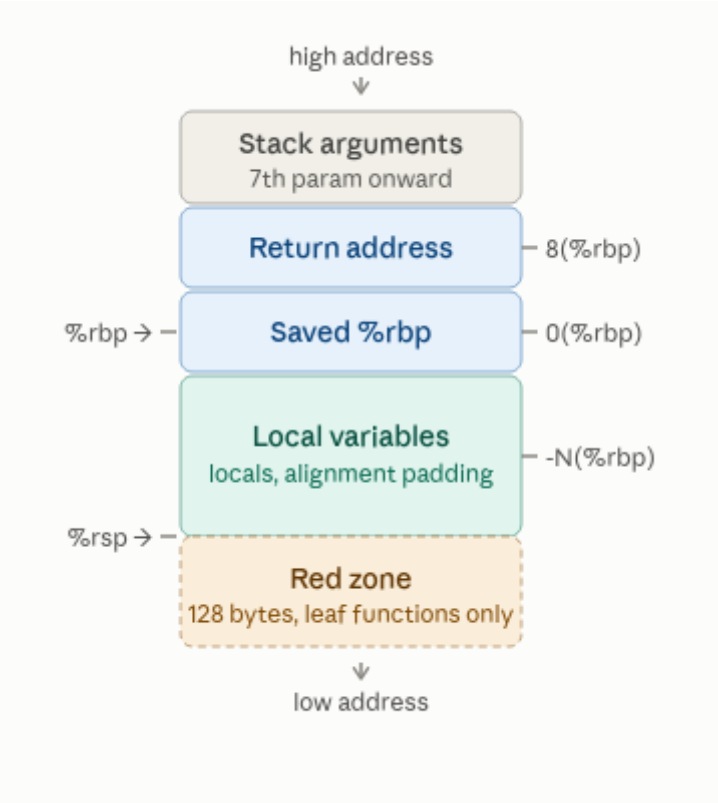
13.4. Red Zone (128 bytes)

This is a distinctive feature of AMD64:

- It's the 128-byte region just below %rsp (lower address than rsp) that a leaf function — a function that doesn't call any other function — is allowed to use temporarily without needing sub %rsp first.
- Since a leaf function doesn't call anything further, that region can't be overwritten by an interrupt or signal handler (per the ABI, signal handlers must also respect the red zone).
- It lets small, simple functions save 1–2 instructions (no need to adjust rsp).

If a function calls other functions (non-leaf), it cannot rely on the red zone, because the call instruction will push a return address that could overwrite exactly that region.

13.5. Stack frame layout



Stack arguments: Sit at the highest address. The first 6 parameters are already in registers (%rdi, %rsi,...), so from the 7th parameter onward they get pushed here before the call instruction runs.

Return address 8(%rbp): The address to jump back to in the caller once the function finishes. The call instruction automatically pushes this address onto the stack (8 bytes, which is why it sits exactly 8 bytes above %rbp).

Saved %rbp 0(%rbp): The push %rbp instruction saves the caller's old frame-pointer value here. The current %rbp officially points to this location.

Local variables -N(%rbp): Holds the function's local variables (plus alignment padding). Since the stack grows downward, accessing a local variable means subtracting a negative offset from %rbp (e.g. -8(%rbp)).

%rsp (Stack Pointer): Marks the current top of the stack (the lowest address officially allocated so far).

Red zone (128 bytes): A "scratch" memory region right below %rsp, reserved for leaf functions to use temporarily without spending an instruction to move %rsp further down.

14. Lab

14.1. Lab

https://github.com/Huyen1411-04/Session-3.git

14.2. Lab 5

ELF Header & File Structure

`readelf` is used to inspect the basic technical information of an executable file (target OS, 32-bit or 64-bit, CPU architecture, etc.).

```
readelf -h memory_layout
```

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x12b0
  Start of program headers:          64 (bytes into file)
  Start of section headers:          14408 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

Meaning:

Identification & architecture group

- Magic: identifying signature (confirms this is a valid executable file)
- Class: 64-bit architecture
- Data: data encoding type
- Version: ELF format version
- OS/ABI: OS interface standard
- ABI Version: ABI version
- Machine: CPU architecture

Execution & security info group

- Type: type of executable (e.g., safety/positioning behavior)
- Entry point address: the address where execution begins
- Flags: CPU processing flags

File structure & size info group

- Size of this header: size of the ELF header
- Size of program headers: size of one Program Header entry
- Number of program headers: number of Program Headers
- Size of section headers: size of one Section Header entry
- Number of section headers: number of Section Headers
- Section header string table index: location of the name table

Looking Up the Symbol Table

`nm` (Name List) lists all functions and variables in the program and shows which memory segment each one resides in.

```
nm memory_layout | grep -E 'global_|static_|dummy_|main'
```

Structure of one Symbol Table line:

```
[ ADDRESS OFFSET ]   [ SYMBOL TYPE ]   [ FUNCTION / VARIABLE NAME ]
```

Convention:

- Uppercase letter → global variable/function
- Lowercase letter → local variable/function

```
0000000000001350 t __do_global_dtors_aux
0000000000003da0 d __do_global_dtors_aux_fini_array_entry
00000000000013a0 T dummy_function
0000000000003d98 d __frame_dummy_init_array_entry
0000000000004014 D global_initialized_var
000000000000401c B global_uninitialized_var
                 U __libc_start_main@GLIBC_2.34
00000000000010e0 T main
0000000000004010 d static_initialized_var
0000000000004020 b static_uninitialized_var
```

Symbol in `nm`	Memory segment	Explanation
T / t	TEXT	Functions containing executable instructions (`main`, `dummy_function`).

Symbol in `nm`	Memory segment	Explanation
D / d	DATA	Global / static variables that ARE initialized (e.g., `= 100`).
B / b	BSS	Global / static variables that are NOT initialized.
U	Undefined	External library functions not yet loaded (`printf`, `malloc`, etc.).

Listing Dynamically Linked Libraries

`ldd` (List Dynamic Dependencies) shows which `.so` (Shared Object) library files your program "borrows" from Linux at runtime.

```
ldd memory_layout
```

```
linux-vdso.so.1 (0x00007d3251ef8000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007d3251c00000)
/lib64/ld-linux-x86-64.so.2 (0x00007d3251efa000)
```

- `libc.so.6`: the system's standard C library (contains the code for `printf`, `malloc`, `free`, etc.).
- `ld-linux-x86-64.so.2`: the Dynamic Loader, responsible for loading libraries into RAM when the program runs.

Disassembling Machine Code into Assembly

`objdump` is used to "dissect" the binary machine code inside the file and disassemble it back into human-readable Assembly language.

```
objdump -d -M intel memory_layout | grep -A 5 "<dummy_function>:"
```

```
00000000000013a0 <dummy_function>:
    13a0:	f3 0f 1e fa          	endbr64
    13a4:	c3                   	ret

Disassembly of section .fini:
```

- Addresses: `13a0`, `13a4`
- Raw machine code: `f3 0f 1e fa`, `c3`
- Assembly instruction: `ret`

Revision #18
Created 2026-08-21 03:09:29 UTC by Huyen Nguyen
Updated 2026-08-28 05:06:26 UTC by Huyen Nguyen